

Season 2 And Season 3 Vision

Goal

Turn the current Season 1 static-data analytics site into a maintainable multi-season analytics platform where Season 2 can be imported historically and Season 3 can be updated shortly after each match.

Product Vision

The ideal workflow:

1. A match finishes.
2. An admin uploads or commits the MKW Table Bot JSON file.
3. The system validates it.
4. The match is added to the canonical dataset.
5. Caches are cleared or rebuilt.
6. The frontend immediately reflects updated analytics.

The person maintaining the season should not need to manually regenerate every CSV, remember deployment quirks, or wait until the season ends.

Recommended Phases

Phase 1: Document And Stabilize Current Season 1

- Keep current API behavior working.
- Centralize frontend API config.
- Fix text encoding/mojibake in labels.
- Add a real extraction command.
- Add basic validation around CSV and JSON input.
- Add a README command for local backend/frontend startup.

Phase 2: Make Season A First-Class Concept

- Add `season` query params to API endpoints.
- Rename data storage around season and division.
- Keep Season 1 URLs working through defaults.
- Add Season 2 CSVs and JSON source folders.
- Update UI copy from "Season 1" to selectable season context.

Phase 3: Build A Deterministic Rebuild Pipeline

- Add one command that rebuilds all CSVs from raw JSON.
- Include duplicate source detection.
- Include a validation report for missing tracks, malformed files, or unusual score counts.
- Add player/team alias mapping files.
- Add tests for extraction and core stats calculations.

Phase 4: Add Live Season 3 Updating

There are two reasonable approaches:

1. Git-based upload: add JSON files to the repo, run a rebuild script in CI, deploy updated data.
2. Admin upload: create a backend upload endpoint and store canonical data outside the repo.

The git-based approach is simpler and more transparent. The admin-upload approach is smoother long term but needs authentication, persistence, and better operational care.

Recommended Near-Term Design

For the next implementation step, prefer a repo-based pipeline:

```
backend/  
├── data/  
│   ├── raw/  
│   │   ├── season-1/  
│   │   ├── season-2/  
│   │   └── season-3/  
│   ├── processed/  
│   │   ├── season-1.csv  
│   │   ├── season-2.csv  
│   │   └── season-3.csv  
│   └── aliases/  
│       ├── players.csv  
│       └── teams.csv  
└── scripts/  
    ├── rebuild_data.py  
    └── validate_match_json.py
```

Then update the API to filter by `season` and `division` instead of loading only `ctc_d{division}.csv`.

Analytics Improvements To Consider

- Overall player averages by season, division, team, and week range.
- Track averages by player and team.
- Head-to-head team matchup suggestions.
- Minimum race controls that are consistent across pages.
- Bagging-aware views instead of changing the player name.
- Player aliases so duplicate names do not split stats.
- Team aliases for renamed tags.
- Match/week filters.
- Trend lines over time for ongoing Season 3.

Operational Requirements For Season 3

- Validation before data becomes visible.
- Clear failure messages when a JSON file is invalid.

- A reliable way to rebuild from scratch.
- A visible "last updated" timestamp.
- Cache invalidation after updates.
- Backups of raw uploaded JSON.

Biggest Architectural Decision Ahead

The main decision is whether Season 3 updates should be file/repo-driven or app-driven.

File/repo-driven is probably best first because the current project is already file-based and static-friendly. App-driven uploads can come later after the data model and validation rules are solid.